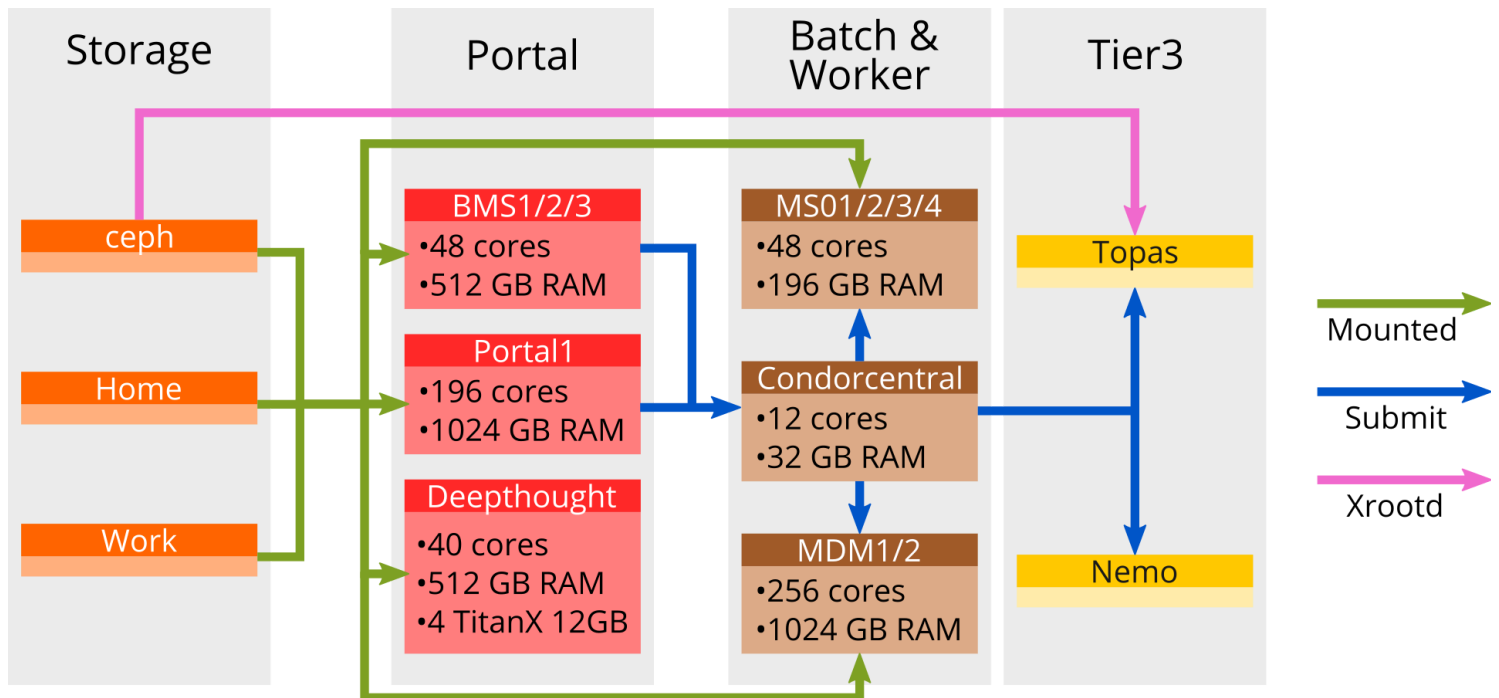


ETP HTCondor Batch System

How To Use the Batch System

A tutorial by Matthias Schnepf is available [here](#) or if you also want to utilize GPUs [here](#).

Local Infrastructure



Submitting a Job with HTCondor

Useful links:

- [Condor manual](#)
- [Condor manual: submit a job](#)
- [Video tutorial](#)

How to Setup VOMS Proxy for Job Submission

For submitting a job that needs access on remote resources or wants to write on the nrg dcache, you need to setup a voms proxy. Your grid certificate is used from the default path (`~/globus`). If you have a .p12 file, you need to do the following [steps](#).

- CMS: `voms-proxy-init -rfc -bits 4096 --voms cms:/cms/country/de --valid 192:00`
- Belle: during setup of gbasf2 `source /cvmfs/belle.kek.jp/grid/gbasf2/pro/bashr` or with `gb2_proxy_init -g belle`

CentOS7 Containers

If you're submitting jobs from the CentOS7 containers (bmsX-centos7.etp.kit.edu or portal1-centos7.etp.kit.edu), please add the following line to your bashrc: `export X509_USER_PROXY=$HOME/.globus/x509_proxy`

Since the containers are using the scheduler of the host machine, a valid proxy needs to be present on the host machine as well. By putting your proxy into your home directory, it is available on every machine.

ETP HTCondor Cluster

There is an HTCondor batch system deployed at ETP, which is one of the most used batch system in HEP. It has access to a wide variety of resources and dispatches your jobs automatically to applicable resources determined by a job description file. Each group of resources – so-called *cloud sites* – supply a uniform environment on each machine, providing identical hardware, technologies, mounts, etc.. Across **different** *cloud sites* we try to provide infrastructure as similar as possible. This necessitates the major distinctions between **local** and **remote jobs** and **CPU-** and **IO-intensive** jobs. Overall we have a few local resources which are specialized for certain tasks, while there's a huge computing cluster in Freiburg available remotely to all physicists. Dispatching your jobs on appropriate resources requires [detailed information on job requirements and attributes](#).

In general, jobs are submitted via the HTCondor command-line interface, but there are tools available in order to simplify the [job submission](#).

Command overview

Command	What it does
condor_submit	Submit new Jobs
condor_status	View Pool Status
condor_q	View Job Queue
condor_q -analyze	Why job/machines fail to match?
condor_ssh_to_job	Create ssh session to active job (useful for debugging)
condor_submit -i	Submit interactive job
condor_hold/release	Hold a job, or release a held job
condor_run	Submit and block
condor_rm	Remove Jobs
condor_prio	Intra-User Job Prios
condor_history	Completed Job Info

For more information, look at the [HTCondor documentation](#).

Job submission

You have to submit your jobs from portal machines. Currently, that's possible from **bms1.etp.kit.edu**, **bms2.etp.kit.edu**, **bms3.etp.kit.edu**, **portal1.etp.kit.edu**, and **deephought.etp.kit.edu**.

To submit jobs with HTCondor, you have to describe them using a JDL file (Job Description Language). This file contains both instructions for the job itself, e.g. which executable and arguments to use, and meta-information, e.g. how much RAM you need, which universe and on which cloud site it should run.

Besides submitting directly from the command line via the [HTCondor command-line interface](#). In the tutorial are some [example files](#) to submit jobs to the ETP HTCondor.

You can also use [grid-control](#) (recommended KIT-CMS version [GitHub](#)), which handles the JDL file creation and job submission for you. Additionally, grid-control adds a huge array of parameters, file transfer, processing, and convenience options. Using grid-control, you can also submit to other batch systems and resources, e.g. NAF. Refer to the tool's documentation for more information.

When submitting directly via the HTCondor command-line interface, we advise you to use the [JDL-Creator](#) for easier creation of job description files.

You can find more examples of JDL file configs on the [HTCondor Job Submission examples](#) page.

You can start an interactive job via `condor_submit -interactive submit.jdl`. When that job starts, you get directly SSHed into the job, and in the same environment, your job starts. The interactive jobs end when you exit the SSH session. This is a good opportunity to develop or debug code.

Job Environment

All jobs running in the ETP HTCondor run inside a container. This enables to run your jobs on each resource in the same software environment. Currently, we provide SLC6, SLC7, CS8 and AlmaLinux9 environments. The standard environment is the SLC7 environment. If your job needs another environment provided by another docker image you can also define it via the [ClassAd](#) (example for SLC7 environment):

```
1 Universe = docker
2 docker_image = mschnepf/slc7-condocker
```

In grid-control via

```
1 JDLData = ... docker_image=mschnepf/slc7-condocker
```

If you use some special libraries, we recommend compiling against libraries provided by cvmfs (e.g. `/cvmfs/sft.cern.ch/...`). The docker container has most of the installed libraries on the portal machines, but not all.

Recommended docker images:

OS	Image name
----	------------

SLC7	mschnepf/slc7-condocker cverstege/cc7-gridjob (updated version, can be used when having problems with remote storage access with the mschnepf/slc7-condocker image)
RHEL8	mschnepf/cs8-base
ALMA9	cverstege/alma9-gridjob

Accounting groups, job & user priority

Currently, we have different accounting groups. The *cms* group consists of 3 subgroups:

- belle
- cms
 - cms.top
 - cms.higgs
 - cms.jet
 - cms.production (for workflows which run longer than 2 days, such as Monte-Carlo production)
- beamdump

This group has to be set upon job submission by setting the value

```
1 accounting_group=...
```

in the jdl-file, respectively by changing your grid-control configuration accordingly:

```
1 JDLData = ... accounting_group=...
```

In general, machines are allocated to users based upon a **user's** (or a group's) **priority**. A lower numerical value for user priority means higher priority, so a user with priority 5 will get more resources than a user with priority 50. User priorities in HTCondor can be examined with the `condor_userprio` command.

Users can give their own jobs a **job priority** which controls which jobs should be processed earlier.

See the [HTCondor Manual](#) for more detailed information.

Remote and local resources

The local ETP resources are:

- meagaschnepfs ms*.etp.kit.edu (cloud site ="schnepf") are dedicated WN which provide 192 CPU-cores with 4 GB RAM per CPU-core.
- mdm machines mdm*.etp.kit.edu (cloud site="mdm") are newer machines with up to 256 CPU-cores per machine and 4GB RAM per CPU core

Regarding remote resources, we primarily distinguish between scientific resources and commercial providers:

- [BwForCluster Nemo](#) is a huge cluster at the University of Freiburg, offering ~8000 slots, which can be used for CPU and IO-intensive Jobs. By default, remote jobs run on this resource. When you use the NEMO cluster, please register your KIT account for NEMO.
- [TQOpAS](#) (Throughput Optimized Analysis System) is a cluster at [GridKa](#) which runs jobs from ETP (cloud site="topas"). The cluster provides 1570 CPU-Cores with about 3GB RAM per Core and 200 Gbit connection to the [GridKa](#) network (including dCache and a gateway to the internet and LHCOne network). It also includes a worker node with 32 NVIDIA V100 which will be available via ETP HTCondor.

Remote resources should be used with some points and limitations in mind:

- Add [the following acknowledgement](#) to your work:

- | | |
|---|---|
| 1 | Acknowledgement : |
| 2 | The authors acknowledge support by the state of Baden-Württemberg through bwHPC and the |

If possible please also send information about your work to [the bwHpc team](#), stating the following values:

- cluster facility: bwForCluster NEMO
 - RV acronym (ask your group leader, e.g. bw16A000)
 - author(s)
 - title or booktitle
 - journal, volume, pages or editors, address, publisher
 - year
- **All input and output data has to be transferred via Grid protocols or HTCondor file transfer.** Please, do **not** transfer files bigger than 100MB via HTCondor.
 - You may get **a lot** of resources there if the cluster is currently underused. This, in turn, can put a huge strain on connected resources. The network connections and HEP storage facilities rarely can handle >3000-4000 jobs doing exactly the same thing simultaneously.
 - The Nemo Cluster is 40 Gbit external connection. The connection Freiburg Karlsruhe is currently limited to 100 Gbit and the handover to the national network (in Karlsruhe), providing access to data from Hamburg and Aachen is limited to 10 Gbit.
 - Remote jobs have to integrate in "foreign" environments, thus we require you to give an upper estimate for the runtime by setting the HTCondor [ClassAd](#) "RequestWalltime". This way we can assure, that only machines which are available for the job runtime will start running that job. Don't forget that the jobs themselves also need the "RemoteJob = True" [ClassAd](#).

Detailed job requirements and attributes

In order to run your jobs on appropriate resources, it is advisable that you define some job requirements and/or attributes. Here we describe, how this is done in JDL files. If you're using grid-control, some of these values are taken directly from the configuration file, others have to be entered identically in the `[condor]` section. HTCondor distinguishes between attributes and requirements:

Requirements

Requirements restrict the applicable machine pool by introducing restrictions ("only run my job on machines which ...").

Restricting this pool can also **positively** influence match-making/priority: Machines that are well equipped to handle IO-intensive jobs will prefer them over jobs that only require CPU.

An example requirement is the need of access to local ETP (storage) resources. You can set this requirement with

```
TARGET.ProvidesETPResources
```

Requirements are identical for Grid-Control and HTCondor.

<u>Requirements</u>	<u>Comment</u>
ProvidesIO	for IO-intensive jobs
ProvidesCPU	for CPU-intensive jobs
ProvidesETPResources	ETP <code>/work</code> , <code>/home</code> & <code>/ceph</code> are available

Attributes

Attributes have additional information about the job, for example, if it's a remote-capable job (because you handle input/output via remote storage or HTCondor's built-in file-transfer). In contrast to requirements, attributes supply *optional* information that **can** be used by the batch system for match-making. Usually, they allow you to access additional resources and/or match you better to the appropriate resources where your job will run in the most appropriate environment.

Grid-control uses the configuration's values translated to HTCondor [ClassAds](#) for the "Request..." class. This means you don't have to explicitly define them in the `[condor]` configuration part.

We also provide multicore jobs. For that set the [ClassAd](#) `RequestCPUs` to the requested amount of CPU-cores for your job. We optimized our HTCondor system for single (default) and eight CPU-core jobs.

Remote job capability has to be explicitly defined by you. If it's not, the job will never run on remote resources. If you do define Remote Job capability, you **must not** require `TARGET.ProvidesETPResources`.

<u>Attribute</u>	<u>Comment</u>
+RemoteJob	Job can (also) run on non-ETP(-local) resources
+RequestWalltime	maximum runtime of one job (in seconds)
RequestMemory	maximal needed memory of one job (in MB)

GPU Jobs

The ETP batch system also provides GPUs. You can request GPUs for your job via `RequestGPUs = NUM_GPUs`. Currently, the batch system provides up to 8 GPUs per job. Please be aware that you also need `+RemoteJob = True`, since external resources provide the GPUs.

HTCondor sets the environment variable `NVIDIA_VISIBLE_DEVICES` to define which GPU is provided to your job. Please do not overwrite it. Some programs can not handle the short UUID format that HTCondor uses. Therefore, please check that your code

can handle that and uses the correct GPUs.

You can select the type of GPU by adding the `GPUs_DeviceName` ClassAd to the `requirements` in your JDL, e.g. `requirements = TARGET.GPUs_DeviceName =?= "Tesla V100-PCIE-32GB"` or the minimal amount of GPU memory (`GPUs_GlobalMemoryMb`), e.g., `requirements = TARGET.GPUs_GlobalMemoryMb >=3000` .

GPUs at TOpAS:

GPU Type	GPUs_DeviceName Classad	#WNs	#GPUs per WN
NVIDIA V100 (32GB)	Tesla V100-PCIE-32GB	1	8
NVIDIA V100s (32GB)	Tesla V100S-PCIE-32GB	2	8
NVIDIA A100 (40GB)	NVIDIA A100-PCIE-40GB	3	8

Run several jobs on one GPU

Thanks to the work of Tim Voigtländer, we are able to run several jobs on one physical GPU. By default, you always get a physical GPU. When you know that you need less than most of the GPU memory and one GPU, you can request a given amount of GPU memory (in MB) via `+RequestGPUMemoryMB = GPU_MEMORY` . Condor fills the GPU memory with other jobs, which enables us to run several jobs on a physical GPU automatically.

GPUs at NEMO2

In the ETP HTCondor batch system also GPUs at NEMO2 are available. These include 8 (out of 9 x 4) NVIDIA L40S and jobs run on them automatically as long as `+RemoteJob = True` and `RequestGPUs = 1` are set during job submission.

To specifically target NEMO2 GPUs, additionally, the cloud site has to be specified in the job requirements `TARGET.CloudSite =?= "BwForCluster-Nemo2"` .

Be aware that NEMO2 doesn't have `/ceph/srv/` available like TOpAS. If you need access to it, make sure your jobs specifically run on TOpAS or add `TARGET.ProvidesEtpCeph =?= True` to the job requirements.

Monitoring

- [ETP HTCondor monitoring](#) for ETP resources (ETP basement, NEMO)
- [TOpAS HTCondor monitoring](#) for TOpAS CPU and GPU
 - You can get an overview of the available free GPU resources via `condor_status -pool f03-001-140-e.gridka.de -constraint 'PartitionableSlot && TotalSlotGPUs>0 && State =?= "Unclaimed"' -af:t Name CPUs MEMORY DISK GPUs`

Launching Dask on the ETP HTCondor Cluster

[Dask](#) is a Python library that can be used for parallel and distributed computing. To run it on an HTCondor cluster, you should first contact the admins to open a port for you. Otherwise, the firewall will block your processes, resulting in OS time out errors. Then you have to define the cluster using [HTCondorCluster](#). Here is a working example for our ETP HTCondor cluster:

```
1 cluster = HTCondorCluster(  
2     scheduler_options={  
3         "port": "3719",
```

```

4   },
5   cores=cores,
6   memory=memory,
7   disk=disk,
8   log_directory=f"{repo_directory}/logs",
9   death_timeout=180,
10  job_script_prologue=job_script_prologue,
11  job_extra_directives = {
12      "Universe":"docker",
13      "docker_image":"mschnepf/slc7-condocker",
14      "docker_network_type":"host",
15      "transfer_input_files": f"{repo_directory}/utils,{repo_directory}/skimmer,{repo_directory}/anal
16      "accounting_group":"cms.higgs"
17  }
18  \

```

Note that while some of the attributes can be directly given to HTCondorCluster, others like your environment, input files or accounting group have to be passed through `job_extra_directives`. Make sure you add the port the admins opened for you to the `scheduler_options`, as shown above. Note that the port will only be open until the next restart of the firewall or the machine, so contact the admins in case it stops working.

Contact

In case of any problems, you can use the mattermost channel [ietp-htcondor](#) to ask for help.

Further contacts:

- Lea Reuter
- Matthias Schnepf

FAQ

- How can I rerun held jobs?

Check first the hold reason of your held job with `condor_q jobID -af HoldReason` and change some [ClassAds](#) with `condor_qedit jobID ClassAd newValue`. A hold job can be released with `condor_release (jobID / user)`

If the hold reason is "Job exceeded maximum number of execution attempts." you have to set some variables for that hold jobs. To release these jobs, run this script: [release_jobs.sh](#)

- I haven't found any policy listings. Where are they?

The cluster uses a global soft-limit fair-share model that ensures equal resource distribution in a limited time span. If the cluster is unused, you get all available resources. If the cluster is in use by others, the cluster is split according to your recent usage: the more resources you got, the fewer resources you will get.

- How can I start multi-core jobs?

Our HTCondor instance has a different kind of worker nodes. All worker nodes support minimal 4 core jobs. To run a job with more cores set in the JDL file "request_cpus = <number of cores>". Please make sure that your program does not run on more than the requested cores!

- Why don't my jobs start running?

Condor decides when and where to run your jobs based on job attributes, such as desired wall time or memory, and worker node attributes, such as available memory or policies. To find out why your job is not starting, run `condor_q -analyze`. If any jobs are broken, run `condor_q -held` to read their error message.

- What does my running job currently?

You can SSH into the job environment with the command `condor_ssh_to_job JobID`". In that environment, you can look inside the logs or see the resource usage via `top`. This works only with jobs that run at the ETP.

- My jobs are executed on machines without access to my home directory /work, /ceph, /storage...! What can I do?

By default, your jobs are eligible to run on different resources. Either try to make your jobs portable, e.g., by pulling configurations and pushing output to the grid. You can disable submission to specific resources; grid control users may add the following to their configuration:

```
1 [condor]
2 JDLData =
3     Requirements=(TARGET.ProvidesETPResources==True)
```

For manual submission, add it directly to your JDL.

```
1 Requirements=(TARGET.ProvidesETPResources==True)
```

For advanced requirements, consult the [ClassAd](#) reference.

- I cannot source CMSSW

try using directly `eval `scramv1 runtime -sh`` instead of `cmsenv`

- My job has the hold reason `Job exceeded maximum number of execution (6) attempts.` How can I release him?

At the ETP HTCondor, jobs get to hold when they start 6 times. You can release these jobs again after resetting some values. There is also a [script](#) that resets the necessary ClassAds and releases the job again. As a parameter for the script, you can use either your username or the job ID, for example, `./release_jobs.sh 1234` or `./release_jobs.sh mschnepf`.